PlotTwist

Dossier de lancement — Rendu R1

Nourane Hammouda · Chaïma Mabrouki · Ornela Mothu · Zaineb BenMahmoud 20 octobre 2026

Formation : Master 1 MIAGE Apprentissage · année universitaire 2026–2027
Version du document : 1.2 · **État du projet :** lancement
Dépôt principal : https://github.com/nourane-hammouda/PlotTwist
Page de projet : à publier ; URL à ajouter dès qu'elle sera disponible.

Équipe, comptes développeurs et dépôt

PlotTwist est porté par quatre étudiantes de M1 MIAGE. Les quatre membres sont réunies dans la même équipe ; aucun binôme ni étudiant isolé n'est laissé de côté.

Membre	Compte développeur GitHub	Adresse configurée pour les commits
Nourane Hammouda	@nourane-hammouda	nouranehamouda09@gmail.com
Chaïma Mabrouki	@Chaimambk	chaima.mbk21@gmail.com
Ornela Mothu	@ornelamt	ornelamothu4@gmail.com
Zaineb BenMahmoud	@zainebbmn	zeyneb.b.mah04@gmail.com

Chaque membre configure Git avec le nom et l'adresse associés à son compte GitHub. Les commits et les pull requests permettent de suivre les contributions individuelles. Le dépôt principal héberge le back, l'API, le front et ce dossier dans `/doc` ; aucun dépôt front séparé n'est prévu à ce stade.

Projet et caractère innovant

PlotTwist est une application qui **génère des enquêtes policières interactives**, puis permet au joueur de les résoudre. Le système assemble personnages, relations, lieux, événements, mobiles, témoignages, indices et fausses pistes. Une enquête courte (environ quinze minutes) vise quatre à six suspects, quatre à six lieux et huit à douze indices.

Le projet ne se limite pas à afficher des récits préécrits ni à gérer des données. Son cœur est un moteur qui construit une enquête **cohérente et résoluble** sous contraintes : le coupable doit avoir un mobile et la possibilité d'agir ; les déplacements doivent respecter la chronologie et le graphe des lieux ; les témoignages et les indices ne doivent pas se contredire ; plusieurs pistes doivent rester plausibles avant l'enquête. Les dialogues seront structurés autour d'intentions prédéfinies, et non d'un chatbot libre.

La dimension innovante et algorithmique porte sur la génération sous contraintes, les graphes de relations et de lieux, la cohérence spatio-temporelle, la détection de contradictions et le classement des suspects. Chaque feature majeure devra exposer un algorithme identifiable, testable et accompagné de ses invariants et cas limites.

Scénarios d'usage

1. **En tant que joueur**, je lance une nouvelle partie et choisis des paramètres simples, afin d'obtenir une enquête différente à chaque partie.

2. **En tant que joueur**, j’explore les lieux et collecte des indices, afin de construire mes propres pistes.
3. **En tant que joueur**, j’interroge un suspect sur son alibi, ses relations ou un indice, afin de confronter son témoignage aux faits connus.
4. **En tant que joueur**, je formule une accusation en désignant un coupable, un mobile et des preuves, afin d’obtenir un résultat expliqué.
5. **En tant qu’enseignant ou démonstrateur**, je démarre une enquête jouable en soutenance, afin de vérifier qu’elle peut être résolue sans connaître sa solution à l’avance.

Plan des features

À chaque itération, **une feature algorithmique est portée par chaque membre**. Chaque feature est développée de bout en bout et touche la base de données, le back, l’API REST et le front ; elle n’est pas répartie par couche technique. Les axes ci-dessous constituent le découpage fonctionnel proposé. L’attribution nominative des features aux membres reste à valider collectivement avant la v0.1.0.

Version cible	Génération de l’enquête	Chronologie et déplacements	Indices et déduction	Interactions et résolution
v0.1.0 · 24/11	Création des personnages et relations — génération de graphe	Chronologie initiale cohérente — contraintes temporelles	Génération et placement d’indices — sélection sous contraintes	Accusation et vérification — comparaison à la solution
v0.2.0 · 08/12	Choix du coupable et du mobile — filtrage de contraintes	Graphe des lieux et trajets — plus courts chemins	Classement des suspects — score de plausibilité	Interrogatoire structuré — confrontation témoignage/indice
v1.0.0 · 19/01	Validation globale et régénération — vérification d’invariants	Détection d’incohérences spatio-temporelles — contraintes croisées	Aide adaptative — sélection d’indice selon la progression	Score et bilan explicatif — évaluation de l’accusation

La v0.1.0 vise une boucle jouable minimale ; la v0.2.0 approfondit les contraintes et prévoit la conteneurisation ; la v1.0.0 vise la résolubilité, l’aide au joueur et la stabilisation. Le périmètre sera ajusté selon les résultats des itérations.

Architecture et frameworks envisagés

L’architecture suit le flux imposé **front → API REST → back Java → base de données**. Le front ne communique qu’avec l’API et n’accède jamais directement à la base.

```
Front web (framework à choisir)
       | HTTP / JSON
API REST — Spring Boot / Spring MVC, OpenAPI
       | cas d’utilisation
Back — Java 21, services et algorithmes indépendants du framework
       | Spring Data JPA / Hibernate
Base — PostgreSQL, schéma versionné par Flyway
```

Couche / besoin	Choix envisagé	Responsabilité
Front	Application web ; React, Vue.js ou Thymeleaf à décider	Partie, exploration, carnet d’indices, interrogatoire, accusation et résultat
API REST	Spring Boot 3, Spring MVC, OpenAPI / Swagger	Endpoints versionnés sous `/api/v1`, validation et erreurs structurées
Back	Java 21, Maven	Règles métier et algorithmes ; la logique reste hors des contrôleurs
Graphes	JGraphT	Relations entre personnages, graphe spatial et calcul de trajets

Couche / besoin	Choix envisagé	Responsabilité
Base de données	PostgreSQL, Spring Data JPA / Hibernate, Flyway	Persistance et évolution versionnée du schéma
Tests et qualité	JUnit 5, Mockito, AssertJ, Testcontainers, JaCoCo	Tests unitaires et d'intégration, couverture et vérification continue
Livraison	Docker Compose, GitHub Actions	Environnement reproductible et CI ; conteneurisation prévue en v0.2.0

L'entité pivot pressentie est la **partie** (`GameSession`), qui relie l'enquête générée, la progression du joueur, les indices collectés et la résolution. Le modèle sera précisé avec les premières features. Les objets métier et algorithmes seront testables indépendamment de Spring ; migrations et API seront couvertes par des tests d'intégration.

Démarche de réalisation d'une feature

Les étapes s'enchaînent dans cet ordre : **idéation → back → tests → API REST → front**. Une étape commence lorsque les critères de fin de la précédente sont satisfaits.

Étape	Travail attendu	Critère de fin
Idéation	Besoin, scénario, algorithme et complexité, alternatives, données/API impactées et critères d'acceptation ; issue et fiche dans `doc/features/`	Fiche relue et validée par un autre membre
Back	Modèle métier, règles, algorithme, cas d'utilisation et persistance	Code compilé ; règles métier vérifiées
Tests	Cas nominaux et limites, invariants, tests d'intégration	Chaque critère d'acceptation est couvert et les tests passent
API REST	Endpoints `/api/v1`, DTO validés, erreurs structurées et documentation OpenAPI	API documentée et testée
Front	Écrans consommant exclusivement l'API	Scénario jouable de bout en bout et pull request relue

Protocole d'usage de l'intelligence artificielle

L'IA est un **outil d'assistance, jamais l'autrice de la décision métier**. Elle peut aider à explorer des options, expliquer une erreur, suggérer des tests, relire ou reformuler la documentation. Elle ne décide pas de l'architecture ni de l'algorithme central de cohérence. Chaque membre doit comprendre, vérifier, adapter et pouvoir expliquer tout élément livré, notamment en soutenance.

Activité	Assistance possible de l'IA	Responsabilité humaine
Idéation et conception	Critique et comparaison de pistes	Choix, justification et rédaction de la spécification
Algorithmes	Explication et signalement de cas oubliés	Conception, invariants et validation du cœur algorithmique
Code et tests	Premier jet technique ou suggestions de cas	Relecture, exécution, adaptation et vérification des tests
Documentation et CI	Reformulation ou aide à la configuration	Vérification des faits et compréhension des étapes

Il est interdit de commiter du code non compris, de transmettre des secrets ou des données personnelles à un assistant, ou de présenter un contenu généré comme entièrement humain. L'usage significatif de l'IA sera noté dans chaque pull request (outil, objectif, éléments repris, modifiés ou rejetés). La part de l'IA par compétence sera déclarée honnêtement à chaque livraison ; les valeurs et le graphe de suivi restent à renseigner et valider par l'équipe avant le dépôt final.

Annexe — Part de l’IA par compétence

Les valeurs ci-dessous ne sont pas encore mesurées ni validées par l’équipe. Elles sont laissées à renseigner plutôt que déclarées à 0 %, puisque ce dossier a bénéficié d’une assistance IA. Les plafonds sont également à fixer collectivement. L’araignée demandée par le template sera générée à partir de ces valeurs avant le dépôt final, puis actualisée à chaque livraison.

Compétence	Niveau	Part IA (R1)	Plafond fixé par l’équipe
Code	Bas	À confirmer	À fixer
Tests	Bas	À confirmer	À fixer
CI/CD	Bas	À confirmer	À fixer
Rédaction	Bas	À confirmer	À fixer
Débogage	Bas	À confirmer	À fixer
Validation	Haut	À confirmer	À fixer
Algorithmique	Haut	À confirmer	À fixer
Spécification	Haut	À confirmer	À fixer
Conception	Haut	À confirmer	À fixer
Idéation	Haut	À confirmer	À fixer